MASINSKO UCENJE - ZADACI I RESENJA

Master akademske studije Geoinformatika | doc. dr Slobodan Jelic

---

ZADATAK 1 (Polinomijalna regresija formulacija i resavanje optimizacionog problema)

Tekst zadatka:

Posmatramo polinomijalni model za aproksimaciju zavisnosti Y od X: y(x, w) = sum_{j=0}^M w_j * x^j.

1. Za dati skup podataka {(x_i, y_i)}_{i=1}^N definisite funkciju greske Err(w) = (1 / 2N) * sum_{i=1}^N (y_i - y(x_i, w))^2. Objasnite zasto odgovara kriterijumu najmanjih kvadrata.
2. Izracunajte parcijalne izvode dErr/dw_k = 0 za k=0,...,M i formulisite sistem jednacina.
3. Napisite sistem u matricnoj formi (Phi^T * Phi) * w = Phi^T * y.
4. Objasnite kako se w dobija resavanjem sistema.
5. Ispitajte uslove postojanja jedinstvenog resenja u zavisnosti od ranga matrice Phi.

RESENJE ZADATKA 1:

1. Funkcija greske predstavlja prosecnu polovinu kvadratnog odstupanja izmerenih vrednosti y_i od predvidjenih vrednosti y(x_i, w). Minimizacija ove sume kvadrata odgovara kriterijumu najmanjih kvadrata jer penalizuje veca odstupanja.
2. Parcijalni izvod po koeficijentu w_k iznosi: dErr/dw_k = -(1/N) * sum_{i=1}^N (y_i - y(x_i, w)) * x_i^k. Postavljanjem dErr/dw_k = 0 dobija se sistem od M+1 jednacina: sum_{j=0}^M w_j * (sum_{i=1}^N x_i^{j+k}) = sum_{i=1}^N y_i * x_i^k.
3. U matricnoj formi, definisanjem Vandermonde matrice Phi dimenzija N x (M+1) gde je Phi_{ij} = x_i^j i vektora ciljeva y, sistem glasi: (Phi^T * Phi) * w = Phi^T * y.
4. Vektor parametara w se dobija resavanjem sistema linearnih jednacina. Ukoliko je matrica Phi^T * Phi invertibilna, w = (Phi^T * Phi)^{-1} * Phi^T * y. U praksi se koristi numpy.linalg.solve ili numpy.linalg.lstsq.
5. Jedinstveno resenje postoji ako je rang(Phi) = M + 1 (pun rang po kolonama), sto zahteva da je broj podataka N >= M + 1 i da su ulazne tacke x_i medjusobno razlicite. Ako je N < M + 1 ili postoje zavisne kolone, rang je manji od M + 1, pa postoji beskonacno mnogo resenja.

ZADATAK 2 (Implementacija u scikit-learn)

Tekst zadatka:

Napraviti sopstvenu klasu za regresiju koja je kompatibilna sa scikit-learn bibliotekom (poly_regression.py). Klasa PolynomialRegression treba da nasledjuje BaseEstimator i RegressorMixin, da u fit(X, y) konstruise matricu Phi i izracuna w resavanjem sistema (Phi^T * Phi) * w = Phi^T * y, da u predict(X) vraca predikcije i u score(X, y) vraca MSE.

RESENJE I KOD ZADATKA 2:

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, RegressorMixin

class PolynomialRegression(BaseEstimator, RegressorMixin):
    def __init__(self, degree=1, alpha=0.0):
        self.degree = degree
        self.alpha = alpha
        self.coef_ = None

    def _design_matrix(self, X):
        X_arr = np.asarray(X).flatten()
        return np.vander(X_arr, N=self.degree + 1, increasing=True)

    def fit(self, X, y):
        Phi = self._design_matrix(X)
        N, M_plus_1 = Phi.shape
        A = Phi.T @ Phi
        I = np.eye(M_plus_1)
        I[0, 0] = 0.0  # w0 se ne regularizuje
        A_reg = A + self.alpha * I
        b = Phi.T @ y
        self.coef_ = np.linalg.lstsq(A_reg, b, rcond=None)[0]
        return self

    def predict(self, X):
        Phi = self._design_matrix(X)
        return Phi @ self.coef_

    def score(self, X, y):
        y_pred = self.predict(X)
        return np.mean((y - y_pred) ** 2)

    def err_function(self, X, y):
        return 0.5 * self.score(X, y)
```

ZADATAK 3 (Izbor modela)

Tekst zadatka:
Koristeci klasu PolynomialRegression, testirati modele za M in {1, 2, 3, 5, 9} na 10 tacaka iz Tabele 1. Izracunati w, Err(w) i MSE. Predloziti nove tacke X = [600, 900, 1300, 1600] mm, izracunati predikcije i diskutovati overfitting i underfitting.

RESENJE I KOD ZADATKA 3:

```
import numpy as np
import pandas as pd
from poly_regression import PolynomialRegression

data = {
    'X': [520, 610, 700, 820, 910, 1030, 1120, 1250, 1390, 1510],
    'Y': [3.1, 3.8, 4.0, 5.1, 5.3, 6.0, 6.2, 5.9, 5.1, 4.1]
}
df = pd.DataFrame(data)
X, y = df['X'].values, df['Y'].values
X_new = np.array([600, 900, 1300, 1600])

for M in [1, 2, 3, 5, 9]:
    model = PolynomialRegression(degree=M, alpha=0.0).fit(X, y)
    mse = model.score(X, y)
    err = model.err_function(X, y)
    preds = model.predict(X_new)
    print(f"M={M} | MSE={mse:.4f} | Err={err:.4f} | Preds={np.round(preds, 2)}")
```

Rezultati i diskusija:
• M=1: MSE = 0.7485, Err = 0.3742. Predikcije: [4.25, 4.72, 5.36, 5.84] t/ha. (Underfitting - model je suvise jednostavan da opise pad biomase pri vecim padavinama).
• M=2: MSE = 0.0576, Err = 0.0288. Predikcije: [3.62, 5.55, 5.51, 3.53] t/ha. (Dobro opisuje parabolicni trend).
• M=3: MSE = 0.0166, Err = 0.0083. Predikcije: [3.60, 5.39, 5.78, 2.69] t/ha.
• M=5: MSE = 0.0112, Err = 0.0056. Predikcije: [3.62, 5.37, 5.70, 3.65] t/ha.
• M=9: MSE = 0.0000, Err = 0.0000. Predikcije: [3.94, 5.27, 6.01, 77.19] t/ha. (Overfitting - greska na obucavajucom skupu je 0, ali model divlja van opsega obuke i daje nerealnih 77.19 t/ha za X=1600).

ZADATAK 4 (Regularizacija polinomijalne regresije)

Tekst zadatka:
Prosiriti klasu PolynomialRegression da ukljuci L2-regularizaciju (Ridge regresiju). Postaviti optimizacioni problem, resiti ga analiticki i dodati parametar regularizacije alpha. Uporediti rezultate sa i bez regularizacije.

RESENJE I KOD ZADATKA 4:
Minimizira se funkcija: Err_reg(w) = (1 / 2N) * sum_{i=1}^N (y_i - y(x_i, w))^2 + (alpha / 2) * sum_{j=1}^M w_j^2.
Izjednacavanjem parcijalnih izvoda sa nulom dobija se sistem: (Phi^T * Phi + alpha * I) * w = Phi^T * y.

```
import numpy as np
import pandas as pd
from poly_regression import PolynomialRegression

X = np.array([520, 610, 700, 820, 910, 1030, 1120, 1250, 1390, 1510])
y = np.array([3.1, 3.8, 4.0, 5.1, 5.3, 6.0, 6.2, 5.9, 5.1, 4.1])
X_new = np.array([600, 900, 1300, 1600])

for M in [3, 5, 9]:
    for alpha in [0.0, 1e-3, 1.0]:
        model = PolynomialRegression(degree=M, alpha=alpha).fit(X, y)
        preds = model.predict(X_new)
        norm_w = np.linalg.norm(model.coef_)
        print(f"M={M}, alpha={alpha:<5} | MSE={model.score(X,y):.4f} | ||w||={norm_w:.2e} | Preds={np.rou
nd(preds, 2)}")
```

Zakljucak regularizacije:
Dodavanjem parametra alpha > 0 sprecava se ekstremni rast koeficijenata w kod visokih stepena (M=9). Regularizacija smanjuje normu koeficijenata ||w|| i stabilizuje predikcije na novim tackama, cime se eliminise pojava preprilagodjavanja (overfitting).